

chapter17

제17장 실무 베스트 프랙티스(best practice)

16장까지 이 교재는 부품을 하나씩 만들었다 — 수집·품질(1~2장), 아키텍처(3장), 신뢰성(4~5장), 스트림·배치(6~7장), 피처와 그 거버넌스(8·13장), 모델의 등록·배포·감시(9~11장), 부하(12장), 통합(14장), LLMops(15장), 윤리·규제(16장). 이 마지막 장은 새 도구를 도입하지 않는다. 대신 앞 열여섯 장이 실제로 남긴 **실측 증거를 회수해**, 무엇이 잘 되었고(성공 요인), 무엇이 어긋났으며(실패 패턴), 비용을 어떻게 다루고(FinOps), 사고에서 어떻게 배우고(포스트모템(postmortem)·error budget), 이 모든 판단을 누가 상설로 관리하는가(Policy Methodology Lab)의 다섯 관점으로 재조립한다. 이 장은 스스로 새 수치를 만들지 않는다 — 회수와 종합만 한다. 실습 17.1은 앞 장에서 **실제로 겪은** 코드 결함, 계획된 장애 드릴, 용량 한계, 벤더 장애를 증거 JSON에서 읽어 포스트모템 보고서·error budget·재발 방지 계획·조직 운영안을 자동 생성한다. 가상 장애를 임의로 작성하지 않는다는 것이 이 실습의 하드를 준수 지점이다 — 사건은 전부 앞 장에서 발생·기록되었다.

학습 목표

DE-MLOps 프로젝트의 성공 요인과 실패 패턴을 앞 장 실측에서 귀납해 설명할 수 있다.
포스트모템과 error budget으로 운영 안정성의 반복 개선 체계를 설계할 수 있다.
비용·품질·책임성의 균형을 12장 부하 실측과 error budget으로 판단할 수 있다.
Policy Methodology Lab 같은 평가·검증 조직의 역할·산출물·책임 경계를 정의할 수 있다.
사건의 증상만 보고 근본 원인과 탐지 층을 가설로 세운 뒤, 증거와 대조해 어긋난 지점을 설명할 수 있다.

17.1 프로젝트 성공 요인

학습 목표

이 교재의 파이프라인을 “운영 가능”하게 만든 반복 구조를 식별한다.
성공 요인이 특정 장의 우연이 아니라 여러 장에 걸친 패턴임을 이해한다.

직관: 성공은 단일 설계가 아니라 반복된 규율에서 온다

좋은 운영은 한 번의 뛰어난 설계에서 오지 않는다. 같은 규율이 장마다 반복되어 습관이 될 때 온다. 이 절은 앞 장에서 **반복적으로 나타난 구조**만 성공 요인으로 꼽는다 — 한 장에서만 통한 기법은 우연일 수 있으므로 제외한다(체리픽(cherry-pick) 금지). 흩어져 있던 실측을 한자리에 모으면, 이 교재의 파이프라인이 왜 정상 작동했는지가 여섯 개의 반복 습관으로 압축된다.

핵심 개념: 여섯 가지 반복된 성공 요인

테스트 증화(10장): pytest:호스트 실행을 모두 통과한 코드가 컨테이너 스모크에서만 IndexError로 실패했다. 실행 환경이 달라야만 드러나는 결함이 있으며, 그래서 각 층은 생략 가능한 중복이 아니라 독립 게이트다.

전달 보증·먹등의 한 쌍(14장): at-least-once 전달과 먹등 UPSERT를 한 쌍으로 설계해, 중복 재전송을 무해화했다(뒤의 D2에서 물리 28건이 고유 17건으로 수렴).

결정성·바이트 동일 증거(9장~): 증거 JSON을 재실행해도 바이트가 같게 설계해 “무엇이 바뀌면 사고인가”의 경계를 유지했다. 7~16장 전체에 걸친 습관이다.

게이트 vs 감시 구분(11·13장): 품질 규칙(계약 위반 차단)과 드리프트(분포 이동 신호)를 다른 층으로 분리해, 정상 변동을 오류로 차단하거나 명백한 결측을 감시가 놓치는 혼동을 막았다.

손 검산 가능 설계(9·12장): 예측 6.0·MAE 1.0을 수작업으로 검산 가능하게 설계해 전 파이프라인의 스모크 기준으로 삼았고, 12장의 처리량 천장도 Little’s law로 손 검산했다.

부분 실패 격리·복구(14장): 마감과 예측을 분리하고 실패를 pending으로 보존해, 의존 서비스가 중단되어도 유실 0으로 복구했다(뒤의 D3).

표 17.1 성공 요인과 실측 근거(출처 장)

성공 요인	실측 근거	출처 장
테스트 증화	컨테이너 스모크에서만 발현한 IndexError	10장
전달 보증·먹등 한 쌍	중복 재전송 무해화(D2)	14장
결정성·바이트 동일 증거	증거 JSON 재실행 바이트 동일	9~16장
게이트와 감시 구분	품질 규칙 ↔ 드리프트 층 분리	11·13장
손 검산 가능 설계	MAE 1.0·처리량 천장 Little’s law	9·12장
부분 실패 격리·복구	API 중단 중 유실 0(D3)	14장

워크드 예제: 하나의 검산값을 여러 단계에서 확인하기

성공 요인은 예측 6.0이라는 한 숫자를 여러 단계에서 확인하는 방식으로 연결된다. 9장에서 champion(평균 예측기, y 평균 6.0)이 확정됐다. 이 값은 손 검산 가능하다 — 입력과 무관하게 항상 6.0을 내므로 “알려진 입력 → 알려진 출력”이 성립한다. 그래서 10장은 pytest:호스트·컨테이너의 모든 스모크에서 “6.0이 나오는가”를 판정 기준으로 삼았고, 컨테이너가 IndexError로 실패했을 때 “6.0이 나오지 않는다”가 곧 경보였다. 12장은 부하 5시나리오 내내 예측이 6.0을 유지하는지로 부하 중 정확성을 확인했고(failures 0), 14장은 통합 MVP의 예측 9건이 전부 6.0인지로 교차 검증했다. 손 검산 가능 설계는 테스트 증화의 판정 기준으로 사용되고, 결정성 검증은 재실행마다 같은 6.0이 나오는지 확인한다.

현장 관점: 성과는 개인 역량이 아니라 조직 역량에서 온다

“성공은 단일 설계가 아니라 반복된 규율”이라는 이 절의 방향은 이 교재만의 감상이 아니라 현장 연구의 관점과 맞닿아 있다. DORA(DevOps Research and Assessment)의 capability 기반 접근(배경·방법론 차용 — 민간 소프트웨어 전달 조직 대상 연구로 공공·MLOps 직접 선례는 아니다)은, 배포 성과를 개인의 역량이 아니라 **반복 가능한 조직 역량**(자동화된 테스트·배포, 버전 관리, 관찰성, 점진적 릴리스 등)으로 설명하려는 관점이다. 이 관점은 위 여섯 요인을 사람이 아니라 구조에서 찾는 태도와 같은 방향이며, 여섯 요인 중 일부(테스트 증화·게이트 구분 등)는 DORA가 꼽는 역량과도 대응한다 — 다만 이 교재의 여섯 요인과 DORA의 역량 목록이 1:1로 겹친다는 뜻은 아니다. 요점은 성공 요인을 구조에서 찾는 것이 이 교재만의 관점이 아니라, 조직 성과를 반복 가능한 역량으로 보려는 현장 연구의 관점과 일관된다는 것이다.

같은 관점을 민간 팀은 네 개의 지표로 측정한다 — 배포 빈도, 변경 리드타임, 변경 실패율, 서비스 복구 시간(DORA 4대 지표). 위 여섯 요인과의 연결은 직접적이다. 테스트 증화·먹등·결정성을 지키면 잘못된 변경이 배포까지 도달하는 비율이 줄고, 부분 실패 격리·복구를 갖추면 장애에서 돌아오는 시간이 짧아진다. 공공 감리는 대개 이 값을 팀별로 측정하는 대신 “그 구조가 갖춰졌는가”를 검수 항목으로 확인한다. 두 방식은 같은 것을 다른 시점에 본다 — 검수는 배포 전에 구조를, DORA 지표는 배포 후에 결과를 본다.

공공 사례

공공 시스템 감리에서 “이 시스템은 왜 안정적인가”라는 질문에, 단일 담당자의 역량으로 답하면 그 담당자가 떠나는 순간 근거가 사라진다. 위 여섯 요인은 사람이 아니라 **구조**에 답을 둔다 — 증화된 게이트, 먹등 설계, 결정적 증거, 층 구분, 검산 가능성, 격리·복구는 담당자가 바뀌어도 남는다. 검수 질문은 “안정성의 근거가 사람의 기억인가 파이프라인의 구조인가”다. ### 핵심 정리 - 성공은 한 번의 설계가 아니라 여러 장에 반복된 여섯 개의 습관에서 왔다. - 성공 요인은 사람이 아니라 구조 (증화·먹등·결정성·층 구분·검산·격리)에 답을 둔다. - 체리픽을 피하려면 한 장에서만 통한 기법이 아니라 반복된 구조만 꼽는다.

17.2 실패 패턴과 예방 조치

학습 목표

앞 장 실제 결함에서 반복된 실패 패턴을 식별한다.
각 패턴을 근본 원인을 없애는 예방 게이트와 짝짓는다.

직관: 침묵 실패의 계보

이 교재에서 반복해 확인한 실패 유형은 하나다 — **비어 있거나 파손된 결과가 성공 상태로 기록된다**. 4장의 유실, 6장의 폐기, 7장의 정제 drop, 8장의 빈 적재, 9장의 로드 실패, 10장의 컨테이너 즉시 실패가 그 사례다. 공통점은 “쓰기·실행은 성공 로그를 남기고, 파손은 나중에 다른

곳에서 드러난다”는 것이다. 관측성(observability) 관점에서 보면, 침묵 실패는 이벤트가 발생했지만 이를 표시하는 지표·로그·알림이 없는 상태다. 따라서 실패가 발생한 단계에서 게이트가 오류를 기록하고 후속 단계를 중단하도록 설계해야 한다.

핵심 개념: 다섯 가지 실패 패턴과 예방 게이트

침묵 실패(쓰기 성공≠사용 가능): 8장 `materialize`가 0건 적재하고도 에러 없이 온라인 `None`을 냈고, 9장 `runs:/` 등록이 성공 로그를 남긴 뒤 별칭 로드에서 실패했다. 예방은 “쓰기 성공”이 아니라 “사용 가능성”(적재 건수·로드·예측)까지 검증하는 게이트다.

배치 가정 결함: 10장 모듈 레벨 경로 계산이 저장소 깊이를 가정해 컨테이너(/app)에서 `import` 시점에 즉시 실패했다. 예방은 환경 가정을 가드로 방어하고 컨테이너 스모크를 독립 게이트로 두는 것이다.

인증 게이트≠결제(권한) 게이트: 15장에서 OpenAI `/v1/models`가 200을 반환했지만 실제 생성(chat/completions)은 결제 비활성으로 전건 429였다. 예방은 인증이 아니라 **실제 사용 엔드포인트**로 헬스체크하고 멀티벤더 폴백을 두는 것이다.

단일 지점 포화: 12장 단일 워커가 한 코어를 포화시켜 처리량이 천장에 걸렸다. 예방은 용량 계획을 실측 천장으로 세우고 수평 확장 임계를 두는 것이다.

진단 기록 삭제: 10장에서 `--rm`이 크래시 로그까지 삭제해 진단을 지연시켰다. 예방은 진단 기동에서 기록을 보존하고, 감시 루프가 “중단”과 “지연”을 구분하는 것이다.

표 17.2 실패 패턴 ↔ 실측 사례 ↔ 예방 게이트

실패 패턴	실측 사례	예방 게이트	출처 장
침묵 실패(쓰기 성공≠사용)	<code>materialize</code> 0건 적재, <code>runs:/</code> 로드 실패	사용 가능성까지 검증 (건수·로드)	8·9장
배치 가정 결함	컨테이너 경로 <code>import</code> 즉사	환경 가드 + 컨테이너 스모크 독립 게이트	10장
인증≠결제 게이트	<code>/v1/models</code> 200이나 생성 전건 429	실제 엔드포인트 헬스체크 + 멀티벤더 폴백	15장
단일 지점 포화	단일 워커 한 코어 포화	용량 계획 실측 천장 + 수평 확장	12장
진단 기록 삭제	<code>--rm</code> 이 크래시 로그 삭제	기록 보존, 중단/지연 구분	10장

반사실: 게이트가 없었다면

침묵 실패가 왜 위험한지는 게이트를 지웠을 때를 가정하면 분명해진다. 8장에서 적재 건수 확인 게이트가 없었다면, `materialize`의 0건 적재는 온라인 조회가 `None`을 낼 때까지 조용했고, 그 `None`이 서빙까지 전달되어 예측 요청에서야 `TypeError`로 발생했을 것이다 — 즉 결함이 데이터 계층에서 서빙 계층으로 번진 뒤에야 드러난다. 9장에서 로드 스모크가 없었다면, 별칭이 로드

불가인 채로 등록만 되어 배포 시점에야 서빙이 실패했을 것이다. 10장에서 컨테이너 스모크가 없었다면, 모든 사전 게이트를 통과한 이미지가 운영망에 배포된 뒤에야 실패했을 것이다. 세 반사실의 공통 구조가 침묵 실패의 정의다 — **결함이 발생한 곳과 드러나는 곳 사이의 거리가 멀수록 피해가 크다**. 게이트는 이 거리를 좁혀, 결함을 발생 지점 가까이에서 잡는다.

검수 기준을 옮기면 온콜 부하가 줄어든다

민간 운영 조직은 이 거리를 줄이는 일을 인시던트(incident) 관리 도구로 자동화한다 — PagerDuty·OpsGenie·Datadog 같은 도구가 탐지·에스컬레이션·복구 알림을 맡고 사람은 판단만 한다. 공공에서 먼저 바뀌야 하는 것은 도구가 아니라 검수 기준이다. “실행 성공 여부”를 보던 항목을 “사용 가능 여부”로 옮기지 않으면, 어떤 도구를 붙여도 침묵 실패는 여전히 성공으로 집계된다.

게이트의 효과는 당직 담당자의 부담으로도 나타난다. 운영 조직은 온콜(on-call) 담당자가 받는 알림 수, 업무 시간 밖 호출 비율, 건당 대응 시간을 지표로 본다. 침묵 실패를 게이트로 잡으면 놓치는 장애(거짓 음성)가 줄고, 게이트가 “중단”과 “자연”을 구분하면 불필요한 호출(거짓 양성)도 함께 감소한다. 알림을 늘리는 것과 알림의 정확도를 올리는 것은 다른 일이며, 온콜 부하를 줄이는 쪽은 후자다.

공공 사례

공공 서비스 장애의 상당수는 “코드는 정상 로그를 남겼는데 사용자만 안 됐다”는 형태다. 침묵 실패 패턴을 조직이 알고 있으면, 검수 항목이 “실행 성공 여부”에서 “사용 가능 여부”로 바뀐다 — 배포 성공 로그가 아니라 실제 조화·예측이 되는지를 게이트로 확인한다. 검수 질문: 우리 시스템의 게이트는 “돌았다”를 보는가 “쓸 수 있다”를 보는가. ### 핵심 정리 - 실패의 원형은 침묵 실패 — 비어 있거나 파손된 결과에 실패를 알리는 표시가 남지 않는다. - 다섯 패턴은 각기 다른 예방 게이트를 요구하며, 예방은 증상이 아니라 근본 원인을 없앤다. - 게이트의 기준을 “실행 성공”에서 “사용 가능”으로 옮기는 것이 침묵 실패의 핵심 대응이다.

17.3 비용 최적화 원칙

학습 목표

12장 부하 실측을 FinOps 관점(용량 계획·확장 비용)으로 회수한다.
비용·품질의 균형을 과잉 설계 경계와 함께 판단한다.

직관: 비용은 자원 × 시간이고, 부하 실측에서 그 자원의 크기가 드러난다

클라우드 비용은 결국 “얼마나 많은 자원을 얼마나 오래 쓰는가”다. FinOps는 이 비용을 엔지니어링·재무·사업이 함께 데이터로 판단하는 실천이다. 구체 단가는 환경(사업자·리전·계약)에 따라 다르므로 이 절은 단가를 지어내지 않는다 — 대신 12장이 실측한 **자원 소비(처리량·CPU·메모리)**를 비용

판단의 근거로 회수한다. “비용을 줄여라”는 구호가 아니라, 실측된 처리량 천장과 확장 배율이 용량 계획의 숫자를 준다.

핵심 개념: FinOps란 무엇이며 왜 “협업”이 정의에 들어가는가

FinOps는 비용을 깎는 기법의 이름이 아니라 **누가·어떻게 비용을 결정하는가의 운영 모델**이다. 이 정의가 왜 중요한지는 반대 상황을 가정하면 드러난다 — 인프라 비용을 재무 부서만 사후에 집계하면, 정작 자원을 켜고 끄는 엔지니어는 자기 결정이 얼마인지 모른 채 과다 프로비저닝(provisioning)하고, 재무는 청구서가 나온 뒤에야 놀란다. 결정과 비용이 분리되어 있는 것이다. FinOps Foundation의 프레임워크(방법론 차용 — 민간 클라우드 재무운영 표준이며 공공 직접 선례는 아님)는 이 분리를 세 원칙으로 메운다: **팀이 협업한다**(재무·기술·제품·리더가 함께 지출과 가치를 관리), **비즈니스 가치가 기술 결정을 이끈다**(최저 비용이 아니라 단위 경제성), **누구나 자기 사용량을 소유한다**(자원을 켜 사람이 그 비용의 주인). 즉 FinOps의 첫 산출물은 절감액이 아니라 “이 자원은 초당 몇 건을 감당하고 얼마인가”를 엔지니어가 스스로 아는 상태다.

실천은 세 단계를 빠르게 순환한다(FinOps Foundation, Inform→Optimize→Operate). **Inform**(파악)은 사용량·비용·효율 데이터를 모아 단위 경제성과 KPI를 드러내는 단계다 — 12장의 부하 실측(인스턴스당 RPS·CPU·RSS)은 이 Inform 단계에 투입할 기술 효율 입력에 해당하며, 사업자 단가(비용 데이터)와 결합해야 비로소 완전한 Inform 산출물이 된다. **Optimize**(최적화)는 그 데이터로 개선 지점을 찾고, **Operate**(운영)는 개선을 실제 조직 절차로 집행한다. 현장에서 이 순환을 회계 언어로 잇는 장치가 **쇼백(showback)**과 **차지백(chargeback)**이다 — 쇼백은 부서별 사용량·비용을 “보여주기만” 하는 가시화이고, 차지백은 그 비용을 실제로 해당 부서 예산에 “청구”한다. 쇼백은 행동을 유도하되 강제하지 않고, 차지백은 비용 소유권을 회계로 못 박는다. 관찰(shadow 로깅)·재시도·멀티벤더 폴백이 전부 요청당 비용을 부과한다는 이 절의 실측(뒤 표 17.3)에서 쇼백·차지백에 올려야 할 항목이 무엇인지가 드러난다.

핵심 개념: 12장 실측을 비용으로 읽기

12장은 10장 서빙 API에 Locust로 부하를 걸어 다섯 시나리오를 실측했다. 이 수치를 FinOps로 다시 읽으면 세 가지 비용 판단이 나온다.

처리량 천장 = 용량 계획의 기준선: 단일 워커는 한 코어를 포화(CPU peak 106.3%)시켜 처리량이 baseline 2498.3 RPS, spike 2788.5 RPS 근처에서 천장에 걸렸다(약 2500–2800 RPS). 사용자를 20에서 100으로 늘려도 처리량은 오르지 않고 지연만 증가했다 — 부하는 용량을 늘리지 않고 대기를 늘린다(Little’s law). 즉 “인스턴스 하나가 초당 몇 건을 감당하는가”의 답이 이 천장이고, 필요 처리량을 이 천장으로 나눈 값이 필요 인스턴스 수의 하한이다.

수직 확장의 비용 대비 효과: 워커를 1에서 2로 늘리면 처리량이 1.26배로 올랐지만 메모리(RSS)는 312MB에서 610MB로 약 2배가 됐다. 처리량 1.26배를 위해 메모리 2배를 지불하는 셈이며, 선형 미달 확장(CPU 경합·오버헤드)이 실측으로 드러난다 — 무작정 워커를 늘리는 것이 비용 효율적이지 않을 수 있음이 이 숫자로 확인된다.

관찰의 비용: shadow 로깅을 켜면 처리량이 0.68배로 떨어졌다. 관찰(감시·로깅)은 공짜가 아니라 요청당 I/O 비용을 부과한다 — 감시를 설계할 때 그 비용을 예산에 넣어야 한다.

표 17.3 12장 부하 시나리오의 비용 해석(실측 — `ch12_load_report.json`)

시나리오	처리량(RPS)	CPU peak	RSS peak(MB)	비용 해석
baseline(워커 1)	2498.3	106.3%	312.3	단일 코어 포화 = 용량 기준선
spike(사용자 100)	2788.5	106.9%	314.1	부하 늘려도 천장 — 지연만 상승
scale(워커 2)	3153.0	181.3%	610.1	처리량 1.26배에 메모리 2배
io(shadow on)	1702.8	115.4%	311.2	관찰 비용 — 0.68배

워크드 예제: 용량 계획을 수작업으로 계산

실측 천장이 있으면 용량 계획이 추정이 아니라 나눗셈이 된다. 어떤 서비스가 최대 초당 5000건을 받아야 한다고 가정한다. 단일 워커의 실측 천장이 baseline 2498.3 RPS이므로, 필요 인스턴스 수의 하한은 $5000 \div 2498.3 \approx 2.0$ — 여유를 두면 3대다. 만약 워커 2개(scale 시나리오, 3153.0 RPS)를 한 인스턴스에 둔다면 $5000 \div 3153.0 \approx 1.6 \rightarrow 2$ 대지만, 이때 인스턴스당 메모리가 610MB로 2배가 되므로 “인스턴스 2대×2워커”와 “인스턴스 3대×1워커”의 비용을 메모리·코어 단가로 비교해 고른다. 핵심은 이 계산의 분모(천장)가 지어낸 값이 아니라 12장 부하 실측이라는 점이다 — 천장이 실측이면 용량 계획 전체가 검산 가능해진다. Little’s law로도 교차 확인된다: baseline에서 사용자 20·평균 지연 7.6ms이면 처리량 $\approx 20 \div 0.0076 \approx 2632$ RPS로, 실측 2498.3과 같은 자릿수다(폐쇄 루프 오버헤드로 약간 낮음).

과잉 설계도 비용이다

비용 최적화는 자원을 줄이는 것만이 아니라 **불필요한 자원을 짓지 않는** 것이기도 하다. 10장이 명시했듯, 일 단위 민원 예측은 사실 배치로 충분하다 — 대상은 열거 가능하고 피처는 하루 한 번 확정되며 이용자가 1분마다 새 예측을 요구할 이유가 없다. 배치로 충분한 문제에 상시 API·오토스케일링을 세우는 것은 비용과 장애 지점을 스스로 늘리는 과잉 설계다. FinOps의 첫 질문은 “더 싸게”가 아니라 “이 자원이 정말 필요한가”다.

민간→공공 적용: 회계연도가 “측정 후 증설”을 굴절시킨다

민간 FinOps의 강점은 탄력성이다 — 측정해서 부족하면 즉시 증설하고, 남으면 즉시 줄인다 (Operate 단계의 “누구나 사용량 소유”가 이걸 가능하게 한다). 그런데 공공 예산은 대개 회계연도 단위로 사전 편성·집행되므로, “이번 달 부하가 늘었으니 다음 주에 인스턴스를 늘린다”가 절차적으로 막히는 경우가 많다. 이 제약은 FinOps를 무력화하는 것이 아니라 **무계중심을 Operate에서 Inform으로 옮긴다** — 사후 탄력 조정이 어려울수록, 편성 전에 “인스턴스당 감당 처리량”을 실측해 두는 사전 용량 계획(12장식 부하 실측)의 가치가 오히려 커진다. 민간이 “운영 중 조정”이라면 공공은 “사전 측정”에 가깝고, 둘 다 근거는 같은 부하 실측이다. 민간이 이 실측을 쓰는 대표적 방식이 예약 인스턴스(Reserved Instance)·절감 계획(Savings Plan) 구매 판단이다 — 실측

천장으로 상시 부하 구간을 계산해 그만큼을 장기 계약으로 낮은 단가에 확보하고, 피크만 온디맨드로 충당한다. 공공에서는 같은 계산이 조달 규격서의 인스턴스 사양·수량을 정하는 근거가 된다. 계약 형태가 다를 뿐 분모는 같은 부하 실측이다.

쇼백·차지백도 공공에서 변형된다. 민간의 차지백은 부서 손익에 비용을 직접 청구해 소유권을 강제하지만, 공공은 클라우드 비용이 특정 예산 항목·조달 계약에 묶여 부서 간 재배분이 회계 규정과 충돌할 수 있다. 그래서 공공에서 현실적인 1차 도구는 차지백보다 **쇼백** — 부서·서비스별 사용량과 그 근거(실측 처리량·관찰 비용)를 가시화해 다음 회계연도 편성의 입력으로 삼는 것이다. 형평성·감사 요구도 없인다: 어떤 서비스가 자원을 얼마나 쓰는지 감사가 대상이 되므로, FinOps의 사용량 데이터는 비용 관리를 넘어 “공적 자원을 무엇에 썼는가”의 설명 책임 자료가 된다.

공공 사례

공공 클라우드 예산은 상한이 정해진 경우가 많다. “상시 고사양 인스턴스”를 관행으로 두면 예산이 조기 소진되고, 정작 부하가 몰리는 시점에 확장 여력이 없다. 12장식 부하 실측이 있으면 “인스턴스당 감당 처리량”이라는 숫자가 생겨, 예산을 실측 천장 기반으로 배분할 수 있다. 검수 질문: 용량 계획이 경험적 판단이 아니라 부하 실측 천장에 근거하는가. ### 핵심 정리 - 비용 = 자원 × 시간이며, 12장 처리량·CPU·RSS 실측이 자원 판단의 근거다. - 수직 확장은 선형 미달 (1.26배에 메모리 2배)이고, 관찰(shadow 0.68배)에도 비용이 있다. - 과잉 설계(배치로 충분한데 상시 API)를 피하는 것이 비용 최적화의 첫걸음이다. - FinOps는 절감 기법이 아니라 결정·비용을 잇는 운영 모델(Inform→Optimize→Operate)이며, 공공은 회계연도 제약 탓에 사후 탄력보다 사전 실측(Inform)·쇼백에 무게가 실린다.

17.4 포스트모템과 지속 개선

학습 목표

비난 없는(blameless) 포스트모템과 error budget의 개념을 이해한다.
앞 장 실측 카운터에서 error budget 소진을 계산하고 반복 개선 체계로 잇는다.

직관: 사고는 사람의 잘못이 아니라 시스템 개선의 재료다

Google SRE의 포스트모템 문화는 하나의 원칙 위에 선다 — **비난 없이(blameless)**(방법론 차용 — 민간 인프라 신뢰성 공학의 관행으로, 공공 직접 선례는 아니다). 포스트모템은 사고의 기록·영향·조치·근본 원인·재발 방지를 담되, 개인을 탓하지 않는다. 왜 이 원칙이 필요한지는 그 반대의 인과를 따라가면 분명하다 — 포스트모템의 결론이 “누가 실수했다”로 끝나고 그것이 인사상 불이익으로 이어지면, 다음 사고의 당사자는 정확히 무엇이 일어났는지를 축소·은폐할 유인을 갖는다. 그러면 조직은 사고에서 가장 중요한 정보(무엇이 어떻게 실패했는가)를 잃는다. 사람을 탓하면 사고를 숨기고, 시스템을 고치면 사고에서 배운다. PagerDuty의 인시던트(incident) 대응 가이드(현장 가이드 — 민간 SaaS 운영)도 같은 논지를 실무 언어로 반복한다: 잘 설계된 blameless 포스트모템은

팀이 지속적으로 학습하고 인프라와 대응 절차를 반복 개선하게 하며, 포스트모템을 건너뛰면 “같은 실수를 다시 피하는 법”을 잃는다. 이 교재의 각 장이 “실제로 발생한 오류”를 숨기지 않고 본문에 수록한 것이 바로 이 문화의 실천이다 — 오류를 부끄러움이 아니라 게이트의 재료로 다룬다.

핵심 개념: error budget — 완벽 대신 합의된 불완전

100% 신뢰성은 목표가 될 수 없다(비용이 무한대이고 달성도 불가능하다). 그래서 SRE는 **error budget**을 쓴다 — SLO 목표(예: 99.9%)와 실제의 차이가 “허용된 불완전”의 예산이다. 예산이 남으면 새 기능을 배포할 여유가 있고, 예산이 바닥나면 배포를 멈추고 안정화에 집중한다. 중요한 것은 **SLO 목표는 조직이 정하는 값(설계 선택)이지 측정값이 아니라는 점**이다 — 측정하는 것은 소진율뿐이다.

error budget이 구호가 아니라 장치가 되려면 소진 시의 행동이 **문서로 미리 합의**되어 있어야 한다. 이것이 **error budget policy**다(방법론 차용 — Google SRE Workbook의 예시 정책). 정책의 뼈대는 단순하다: 예산이 남아 있는 동안은 개발팀이 배포 속도를 스스로 정하되, 예산이 소진되면 **위험을 늘리는 신규 기능 배포를 동결(freeze)하고 신뢰성 작업에 집중**한다. 단, SRE Workbook의 예시 정책도 긴급·보안 수정(P0/P1급) 같은 예외와 에스컬레이션 경로를 함께 규정한다 — 즉 “모든 배포 금지”가 아니라 “위험을 늘리는 신규 기능의 동결”이며, 예외를 밟으려면 사전에 정한 승인 경로를 거친다. 핵심은 이 결정이 사고 당시의 감정이나 힘의 논리가 아니라 미리 합의된 숫자(소진율)로 자동 발동된다는 것 — “지금 배포해도 되나”라는 반복 논쟁을 예산 잔량 하나로 대체한다. 이 실습(17.1)은 소진을 계산까지만 하고 동결을 집행하지는 않는다. 실무에서는 이 소진율 임계를 CI/CD 파이프라인의 배포 게이트에 연결해 정책을 자동 집행한다.

그러면 SLO 목표를 몇으로 두어야 하는가. 이 값은 측정으로 정해지지 않으므로 선택하는 근거가 따로 필요하다. 세 가지를 본다. 첫째, **사용자가 열화를 알아차리는 지점**이다. 하루 몇 분의 중단을 민원인이 재접속으로 넘기는 조회 서비스와, 한 번의 누락이 곧 피해인 재난 경보는 같은 목표를 쓸 이유가 없다. 둘째, **한 자리 올릴 때의 비용**이다. 99.9%를 99.99%로 올리면 허용 실패가 10분의 1로 줄고(뒤의 워크드 예제), 그만큼 이중화·감시·대기 인력이 늘어난다. 셋째, **의존하는 구성 요소의 상한**이다. 외부 API 하나가 99.5%를 보장한다면 그것을 동기로 호출하는 서비스가 99.9%를 약속할 수 없다. 이 셋을 적어 두지 않고 99.9%를 선택하면 그것은 합의된 목표가 아니라 관행을 답습한 수치이고, 관행을 답습한 목표는 소진되어도 아무도 동결을 집행하지 않는다.

실습 17.1은 앞 장 실측 카운터에서 네 SLO의 예산 소진을 계산했다(`ch17_error_budget.json`).

표 17.4 error budget 실측 소진(앞 장 카운터에서 계산)

SLO	목표(설계값)	실측 카운터	소진	판정	출처 장
요청 가용성	99.9%	172633 요청, 실패 0	허용 172건 중 0	예산 내	12장
예측 성공	95%	예측 12건, 실패 3	원 3건 → 복구 후 0	재시도로 잔여 0	14장
품질 게이트 (MAE)	≤ 1.05	채점 2일, 전체 MAE 1.0	2일 중 1일 초과	전체 통과·국소 초과	14장
LLM 생성 (멀티벤더)	≥ 1 벤더 가용	CLOVA 8, OpenAI 0	OpenAI 전건 소진	벤더 폴백으로 지속	15장

네 예산이 각기 다른 교훈을 준다. **요청 가용성**은 12장 부하 5시나리오에서 172633개 요청이 처리되는 동안 실패가 0이었으므로, 99.9% 목표가 허용하는 172건의 예산을 한 건도 쓰지 않았다 — 부하 중에도 정확성(예측 6.0)이 유지됐다는 뜻이다. **예측 성공**은 원 실패 3건이 95% 목표의 허용치 (0.6건)를 넘겼지만, 그 3건이 전부 14장 D3의 API 중단 건이었고 재시도로 잔여가 0이 되었다 — 순간 소진과 복구 후 잔여를 구분해 기록하는 것이 핵심이다. **품질 게이트**는 전체 MAE 1.0이 게이트 1.05를 통과했지만 일자별로는 2일 중 1일(7/3)이 초과했다 — 전체 지표가 국소 열화를 가릴 수 있다는 감시 입도의 교훈이다. **LLM 생성**은 단일 벤더 예산이 429로 전건 소진됐어도 멀티벤더 폴백 (CLOVA 8건 응답)으로 서비스가 지속됐다.

워크드 예제: error budget을 수작업으로 계산

요청 가용성 예산을 직접 계산한다. SLO 목표를 99.9%로 정하면, 허용되는 실패 비율은 $1 - 0.999 = 0.001$ 이다. 12장 5시나리오의 총 요청은 $27496 + 18751 + 60980 + 34713 + 30693 = 172633$ 건이므로, 허용 실패 예산은 $172633 \times 0.001 = 172.633 \rightarrow$ 내림하여 172건이다. 실측 실패는 0건이었으므로 소진은 0, 잔여는 172건 전부다. 목표를 더 엄격히 99.99%로 올리면 허용은 $172633 \times 0.0001 \approx 17$ 건으로 줄어든다 — SLO를 한 자리 올릴 때마다 예산이 약 10분의 1로 준다는 것이 손 계산으로 보인다. 예측 성공 예산도 같은 방식이다: 총 12건에 95% 목표면 허용 실패는 $12 \times 0.05 = 0.6$ 건인데, 원 실패가 3건이라 순간적으로는 예산을 초과했다($3 > 0.6$). 그러나 그 3건이 전부 D3의 API 중단 건이었고 재시도로 잔여가 0이 되었으므로, 복구 후 소진은 예산 안으로 돌아온다. 이 두 숫자를 나란히 기록하는 것이 “얼마나 실패했나(3)”와 “얼마나 남았나(0)”를 구분하는 실측이다.

포스트모템에서 반복 개선으로

포스트모템은 보고서로 끝나면 의미가 없다. 근본 원인(3-why)에서 재발 방지 게이트를 뽑아 파이프라인에 심고, 재검증하는 순환이 되어야 한다. 이 순환이 8~16장에서 실제로 돌아왔다 — 각 장의 “실제로 겪은 오류”가 곧 그 장의 새 게이트를 낳았다. 이 교재 전체를 하나의 개선 원장으로 압축하면 다음과 같다.

표 17.5 장별 “실제로 겪은 오류 → 재발 방지 게이트” 원장

장	실제로 겪은 오류	낚은 게이트
8장	materialize 침묵 0건 적재	적재 건수 확인 + 온라인·오프라인 일관성 대조
9장	runs:/ 등록 후 로드 실패	등록 직후 로드·예측 스모크
10장	컨테이너 경로 import 즉시 실패	컨테이너 스모크 독립 게이트 + 환경 가드
12장	단일 워커 포화 천장	부하 프로파일로 용량 천장 측정
14장	compose 이중 build 경합	이미지=build 선언 1개 원칙
15장	OpenAI 생성 전건 429	생성 엔드포인트 헬스체크 + 멀티벤더 폴백

실습 17.1이 자동 생성한 재발 방지 계획(ch17_recurrence_prevention_plan.md)이 이 순환을 사건별로 표로 정리한다. 원장이 말하는 것은 하나다 — 이 교재의 파이프라인은 처음부터 완벽해서가 아니라, 매 장의 실패를 게이트로 바꿔 왔기 때문에 운영 가능해졌다.

여기서 두 번째 선택이 나온다 — **어떤 사건을 포스트모템 대상으로 삼는가**. 모든 사건에 보고서를 쓰게 하면 보고서가 형식이 되고 결국 아무도 읽지 않는다. 반대로 기준 없이 “큰 것만” 쓰게 하면 담당자가 크기를 판단하게 되어 축소 보고의 유인이 되살아난다. 그래서 트리거를 미리 확정한다. 실무에서 쓰는 기준은 대체로 셋이다 — ① 사용자에게 보이는 영향이 있었는가, ② error budget을 정해 둔 비율 이상 소진했는가, ③ 탐지가 늦었거나 복구 절차가 없었는가. 셋째가 중요한 이유는 피해가 작아도 **탐지 실패는 다음 사건에서 그대로 반복되기 때문이다** — 8장의 침묵 0건 적재가 그 예다.

포스트모템이 쌓이면 세 가지 값이 따라 나온다. **MTTR**(평균 복구 시간)은 사건마다 기록한 탐지 → 대응 → 복구 타임라인의 평균이다. **SLO 달성률**은 측정 기간 중 목표를 지킨 비율인데, 이 장의 품질 게이트는 채점 2일 중 1일이 초과했으므로 달성률 50%다 — 전체 MAE 1.0이 게이트를 통과했다는 사실과 나란히 놓아야 감시 입도의 문제가 드러난다. 셋째는 **심각도별 사건 분포**다. 민간은 SEV-1(전면 중단)부터 SEV-4(경미한 열화)까지 등급으로 나누는데, 실습 17.1이 사건 8건을 네 유형으로 분류한 것이 그 등급화의 앞 단계다 — 등급을 매기기 전에 원인 계통부터 구분해야 적절한 계층에 재발 방지 조치를 적용할 수 있다.

민간→공공 적용: blameless와 감사 문화의 긴장, 그리고 회계연도

blameless 포스트모템을 공공에 옮길 때 가장 먼저 부딪히는 것은 **감사 문화와의 정면 긴장**이다. 공공 감사는 본질적으로 책임 규명 지향이다 — “누가 무슨 권한으로 무엇을 했고, 그 결정이 규정에 맞았는가”를 따진다. blameless는 정반대로 개인 책임을 일부러 괄호에 넣고 “시스템이 왜 이 실수를 쉽게 만들었는가”를 묻는다. 이 둘은 목적이 다르므로 한쪽이 다른 쪽을 이기는 문제가 아니라 **트랙을 분리하는 설계 문제**다. 실무적 전환은 이렇다: 사고 학습(왜 실패했나·어떤 게이트가 없었나)을 위한 blameless 포스트모템 트랙과, 규정 위반·고의·중과실을 다루는 책임 트랙을 **명시적으로 분리**하고, 전자에서 나온 진술을 후자의 징계 근거로 전용하지 않는다는 규칙을 문서화한다. 이

분리가 없으면 blameless는 선언에 그치고 담당자는 여전히 축소 보고한다 — 학습 트랙의 발언이 언제든지 징계로 되돌아올 수 있다면 아무도 솔직해지지 않기 때문이다. 감사의 정당한 요구(책임성)와 운영의 정당한 요구(학습)를 동시에 만족시키는 길은 둘을 섞지 않는 것이다.

error budget 기반 릴리스 속도 조절도 공공 제약을 통과하며 굴절된다. 민간에서 error budget policy는 “예산 남으면 빨리 배포, 소진되면 동결”로 배포 **속도**를 실시간 조절하는 장치다. 그러나 공공의 배포는 조달 계약·회계연도·감리 일정에 묶여 민간처럼 자유롭게 빨라지거나 느려지지 않는다. 그래서 공공에서 error budget의 일차 가치는 “속도 조절”보다 “**동결 결정의 객관적 근거**”에 있다 — 안정화가 필요하다는 판단을 담당자의 직감이나 상급자의 압력이 아니라 소진율이라는 합의된 숫자로 정당화할 수 있게 한다. 회계연도 말 예산·인력 제약으로 신뢰성 작업을 미루자는 압력이 들어올 때, “예산이 소진됐으므로 정책상 동결”이라는 사전 합의는 그 압력에 맞설 근거가 된다. 민간이 error budget으로 속도를 얻는다면, 공공은 error budget으로 **정당성**을 얻는다.

공공 사례

공공 감사에서 장애 이력은 “누가 잘못했는가”로 흐르기 쉽고, 그러면 담당자는 방어적으로 사고를 축소 보고한다. 비난 없는 포스트모템은 초점을 사람에서 시스템·절차로 옮겨, 사고를 투명하게 드러내고 재발 방지로 연결한다. error budget은 “얼마나 안정적이어야 충분한가”를 정치가 아니라 숫자로 합의하게 한다. 검수 질문: 장애 이력이 비난 없이 기록되고 재발 방지 게이트로 이어지는가. ### 핵심 정리 - 포스트모템은 비난 없이 — 사람이 아니라 시스템·절차를 고친다. - error budget은 SLO 목표(설계값)와 실측의 차이이며, 소진율만 실측에서 계산한다. 소진 시 배포 동결은 error budget policy로 사전 합의해 자동 발동한다. - 포스트모템 → 3-why → 재발 방지 게이트 → 재검증의 순환이 8~16장에서 실제로 돌아왔다. - 공공에서는 학습 트랙(blameless)과 책임 트랙(감사)을 분리하고, error budget을 동결 결정의 객관적 근거로 쓴다.

17.5 Policy Methodology Lab: 평가 기준, 벤치마크(benchmark), 승인 게이트 관리

학습 목표

13·15·16장 거버넌스를 상설 조직의 관점으로 종합한다.

평가 기준·벤치마크·승인 게이트를 관리하는 조직의 역할과 책임 경계를 정의한다.

직관: 흩어진 게이트를 관리하는 상설 조직

13장은 피처 접근 게이트를, 15장은 프롬프트 승인 게이트를, 16장은 배포·영향평가 게이트를 만들었다. 각 게이트는 개별 장에서 작동했지만, 그것을 **누가 상설로 관리하고 개정하는가**는 조직의 문제다. 이 “관리 주체” 자체를 명시적 기능으로 세워야 한다는 요구는 이 교재만의 발명이 아니다 — NIST AI 위험관리 프레임워크(배경·일반원칙 — 이 Lab의 직접 선례는 아니나 공공·민간 공통으로 쓰이는 AI 위험관리 일반 틀)는 네 기능 중 첫째로 **Govern**을 두어, 위험 관리가 프로젝트별

임기응변이 아니라 조직에 상설된 문화·구조·책임으로 자리 잡아야 한다고 본다. Policy Methodology Lab은 그 Govern 기능을 정책 AI 맥락에 구체화한 형태로 읽을 수 있다. 다만 분명히 할 것은, Lab은 정책 결정을 내리는 시스템이 아니라 정책 AI의 **평가 기준·벤치마크·승인 게이트를 관리하는 운영·연구 조직**이라는 점이다. 정책효과 평가를 예측 문제로 단순화하지 않고, “무엇을 어떻게 평가하고 무엇을 통과시키는가”의 기준을 관리한다.

핵심 개념: 역할·산출물·게이트 계보

실습 17.1이 자동 생성한 조직 운영안(ch17_org_operating_plan.md)은 네 역할과 세 승인 게이트로 구성된다.

표 17.6 Policy Methodology Lab의 역할·산출물·근거 장

역할	관리 대상	산출물	근거 장
평가 기준 관리자	정책 AI 루브릭(법적 정합성·중립성·유해성)	루브릭 개정·버전 관리	16장
벤치마크 관리자	그룹별 형평·편향 평가 지표	공정성 평가·모델카드 자동 생성	16장
승인 게이트 운영자	접근·프롬프트·배포 승인	프롬프트 레지스트리 승인(요청자·검토자 분리)	15장
피쳐 거버넌스 담당	카탈로그·접근 로그·변경 승인	감사 점검표 자동 생성· 영향 소비자 식별	13장

표 17.7 승인 게이트 계보

게이트	결정	근거 장
피쳐 접근 게이트	역할×행위 인가 + 목적 제한(PIPA 제18조)	13장
프롬프트 승인 게이트	활성 프롬프트 승인 기록(요청자· 검토자 분리)	15장
배포·영향평가 게이트	모델카드·영향평가·자동화 결정 이의제기	16장

연구 조직과 운영 조직의 책임 경계

Lab이 잘 굴러가려면 두 종류의 일이 분리되어야 한다. **연구 조직**은 평가 기준·벤치마크의 방법론을 설계하고 개정한다(“무엇을 어떻게 평가하나”). **운영 조직**은 그 게이트를 집행하고 로그·감사·재발 방지를 돌린다(“실제로 지켜지나”). 둘을 한 팀이 겸하면, 자기가 만든 기준을 자기가 통과시키는 이해충돌이 생긴다. 특히 중요한 주의 하나 — **LLM-as-a-judge는 보조이며 단독 승인 근거로 쓰지 않는다**. 16장이 명시했듯 채점 주체는 사람·독립 검증이고, 15장이 실측했듯 “근거가 있어도 모델이

답변을 거절"할 수 있으므로 자동 판정을 최종 관문으로 삼지 않는다. 15장의 구체 사례가 이를 뒷받침한다 — 게이트가 생성을 허용한 질의에도 모델이 자발적으로 거절한 경우가 있었다("근거 존재 ≠ 답변 가능"). 판정 주체가 사람·독립 검증이어야 하는 이유는, 모델의 판단이 확률적이고 벤더·버전·시점에 따라 흔들리기 때문이다. Lab은 LLM 판정을 1차 선별(triage)로만 쓰고, 승인·반려의 최종 서명은 사람이 한다 — 이것이 15장 프롬프트 승인의 "요청자·검토자 분리"를 조직 계층으로 올린 형태다. 이 분리를 제도로 굳힌 형태가 민간 금융권의 1선·2선·3선 체계다 — 1선(현업·개발)이 만들고, 2선(리스크 관리·모델 검증)이 독립적으로 검증하며, 3선(내부 감사)이 앞의 둘이 규정대로 돌아갔는지 본다. 규제 산업에서는 이 삼분이 감독 요건으로 강제되는 경우가 많다. Lab의 연구·운영 분리는 그중 1선과 2선의 경계에 해당하고, 3선에 해당하는 감사 기능은 공공에서 이미 조직 밖에 있다(감리·감사원). 즉 공공은 3선을 새로 만들 필요가 없는 대신, 2선이 별도 조직으로 서 있지 않으면 1선이 자기 결과를 자기가 검증하게 된다 — Lab을 상설로 두는 실질적 이유가 여기에 있다.

공공 사례

공공 AI의 평가 기준이 프로젝트마다 제각각이면, 같은 위험(편향·환각·목적 외 이용)이 매번 새로 논의되고 누락된다. Policy Methodology Lab 같은 상설 조직이 루브릭·벤치마크·게이트를 관리하면, 개별 프로젝트는 이미 검증된 기준을 재사용하고 개정 이력으로 추적한다 — 13장 카탈로그가 피처를 재사용 자산으로 만든 것과 같은 논리를 평가 기준에 적용한 것이다. 검수 질문: 평가 기준·벤치마크·승인 게이트가 프로젝트별 임의가 아니라 상설 조직의 관리 대상인가. ### 역방향: 이 장의 운영 틀을 민간 현장에 옮기면

지금까지는 민간 관행(blameless-error budget·FinOps·중앙 거버넌스 조직)을 공공 제약에 맞춰 굴절시켰다. 반대 방향도 성립한다 — 이 장에서 세운 틀을 그대로 민간 현장에 적용하면 무엇이 유지되고 무엇이 바뀌는지 세 현장에서 확인한다.

SRE 조직의 릴리스 관리에서는 서비스마다 SLO를 정하고, error budget이 소진되면 신규 기능 릴리스를 동결한다. 표 17.4의 계산이 그대로 쓰이고, 달라지는 것은 주기와 집행력이다. 민간은 예산 잔량을 실시간 대시보드로 보며 분기마다 소진 원인을 리뷰하지만, 이 실습은 소진을 계산까지만 하고 동결을 집행하지 않는다. 공공에서 동결 결정의 정당성 근거로 쓰던 소진율이, 민간에서는 배포 속도를 조절하는 손잡이가 된다.

계획적 장애 주입은 14장 드릴 3종(오염 격리·중복 흡수·API 중단 복구)이 민간에서 상설화된 형태다. 차이는 장소와 빈도다 — 이 교재는 통합 테스트 환경에서 한 번 주입했고, 카오스 엔지니어링(chaos engineering)은 운영 환경에서 상시로 인스턴스를 중단시키거나 네트워크를 차단한다. 표 17.9가 드릴과 실제 결함을 분류로 구분한 이유가 여기서 분명해진다 — 주입을 상시로 하면 사건 목록의 대부분이 드릴이 되고, 그것을 실제 사고로 집계하면 신뢰성 지표가 통째로 왜곡된다.

플랫폼 팀의 모델 거버넌스는 표 17.6의 네 역할을 기업 조직도로 옮긴 것이다. ML 플랫폼 팀이나 AI 전담 조직이 모델 레지스트리·배포 게이트·감시·재훈련을 중앙에서 관리하고, 개별 팀은 그 게이트를 통과해야 배포한다. 관리 대상은 다르다 — Lab은 정책 AI의 평가 기준(법적 정합성·중립성·형평)을, 플랫폼 팀은 모델 품질·비용·SLO를 관리한다. 게이트를 설계하는 사람과 게이트를 통과시키는 사람을 나누는 원칙은 양쪽에서 같다.

인시던트 대응 절차와 런북(runbook — 장애 유형별 대응 절차를 적은 문서)도 그대로 옮겨진다. 표 17.5의 “실제로 겪은 오류 → 낳은 게이트” 원장이 런북의 원재료다 — 무엇이 실패했고 어떤 게이트가 그것을 잡았는지가 런북의 본문이 된다.

공통 원칙: 사건을 원인 계통으로 분류하는 것, SLO 목표를 설계값으로 확정하는 것, 소진 시의 행동을 사전에 합의하는 것, 게이트 설계자와 집행자를 분리하는 것 — 이 네 결정은 도메인과 무관하게 똑같이 내려야 하고, **근거만 도메인에 따라 달라진다**. 공공에서 그 근거가 감사·형평성·법적 의무라면, 민간에서는 매출·비용·SLA다. 방향은 여기서도 비대칭이다 — 공공의 증거 의무를 민간으로 가져가면 그대로 자산이 된다. 재발 방지 원장은 런북이 되고, 자동 생성한 운영 개선 체크리스트는 신규 인력 인수인계 문서가 되며, 결정적 증거 JSON은 감사 자료 대신 회귀 판정의 기준선이 된다.

핵심 정리

Policy Methodology Lab은 평가 기준·벤치마크·승인 게이트를 관리하는 상설 조직이다. 13·15·16장의 게이트가 네 역할과 세 승인 게이트로 종합된다. 연구(기준 설계)와 운영(게이트 집행)을 분리하고, LLM-as-a-judge는 보조로만 쓴다.

실습 17.1 장애 포스트모템 보고서 작성(★★★☆☆)

이 실습은 **가설 → 실행 → 대조 → 위험 식별** 네 단계로 진행한다. 순서에 실무의 이유가 있다. 현장에서 포스트모템을 쓰는 사람은 증거를 다 모은 뒤에 원인을 적는 것이 아니라, 증상을 보고 원인을 먼저 짐작한 다음 그 짐작을 로그로 검증한다. 추정이 어긋나는 지점이 곧 자기 시스템 이해의 결함이고, 그 결함이 다음 사건에서 탐지를 늦춘다. 그래서 증거 JSON을 열기 전에 가설을 먼저 기록하고, 실행 결과와 대조한 뒤에 검수 언어로 옮긴다.

실습 목표

사건의 증상만 보고 근본 원인과 탐지 층을 **먼저 가설로 적는다**(가설).
앞 장 실제 장애 증거(JSON)를 읽어 포스트모템 서식으로 구조화한다(실행).
error budget·재발 방지 계획·운영 개선 체크리스트·조직 운영안을 자동 생성한다(실행).
가설과 생성된 증거를 대조하고, 어긋난 지점이 무엇을 가리키는지 설명한다(대조).

입력 데이터

이 실습은 새 장애를 만들지 않는다. 입력은 전부 앞 장에서 **실제로 발생·기록된** 사건이다 — ① 8·9·10장 집필 중 실제로 겪은 코드 결함 3건(incident_catalog.json, 각 항목이 해당 장 원고 위치를 source로 인용), ② 12장 부하 실측(ch12_load_report.json, 용량 한계), ③ 14장 장애 드릴 3종(ch14_integration_report.json, 계획된 주입), ④ 15장 멀티벤더

LLM(`ch15_llm_snapshot.json` , 벤더 장애). 드릴(계획된 주입)과 실제 결함, 용량 한계, 벤더 장애를 분류로 명확히 구분해 통합하지 않는다. 인용 수치는 모두 실제 실행 산출물 (`ch17_postmortem_report.json`)에서 산출된다.

1단계 가설 — 증거를 보기 전에 근본 원인을 적는다

스크립트를 실행하기 전에 아래 표를 채운다. 네 사건의 **증상만** 제시했다. 17.2의 표 17.2와 17.4의 표 17.5에 답이 적혀 있으므로, 그 두 표를 참조하지 않고 증상만으로 작성한다 — 실행 뒤에 비교하는 것이 이 단계의 목적이다.

표 17.8 사건 원인 가설표 — 학습자 작성용

사건(증상만)	내 근본 원인 가설	어느 층에서 잡혔어야 하는가	대조 결과(3단계 기입)
온라인 피쳐 조회가 에러 없이 None을 반환한다			
모델 등록은 성공했는데 별칭 로드에서 설정 파일을 찾지 못한다			
컨테이너가 <code>/health</code> 에 응답하지 않는다(호스트 실행은 정상)			
인증 헬스체크는 200인데 생성 요청은 전건 실패한다			

작성할 때 지킬 것이 둘이다. 첫째, “설정 실수”나 “환경 차이” 같은 포괄적 표현을 쓰지 않는다 — 무엇을 무엇으로 잘못 가정했는지까지 적는다. 둘째, 셋째 열에는 조직·사람이 아니라 **파이프라인의 층**을 적는다(예: 적재 직후 / 등록 직후 / 컨테이너 스모크 / 실제 사용 엔드포인트). 이 열이 뒤에서 재발 방지 게이트의 위치가 된다.

이어서 설계 선택 두 가지를 정하고 근거를 적는다. 실행 결과가 나오기 전에 채워야 대조가 성립한다.

항목	내가 정한 값	근거(무엇에서 나왔는가)	실측 대조(3단계 기입)
요청 가용성 SLO 목표			
배포 동결을 발동할 소진 비율			
포스트모템을 의무로 쓸 사건의 기준			8건 중 몇 건이 해당하는가

근거 칸에 “업계 표준이라서”를 쓰지 않는다. SLO 목표는 17.4의 세 기준(사용자가 열화를 알아차리는 지점·한 자리 올릴 때의 비용·의존 구성 요소의 상한)에서 선택하고, 포스트모템 기준은 “8건 전부에 보고서를 쓰면 아무도 읽지 않는다”는 제약을 전제로 정한다.

2단계 실행 — 앞 장 증거를 읽어 보고서 생성

Python 3.10 이상(이 실습은 3.13에서 검증)이면 충분하고 외부 인프라·의존성이 없다 — 앞 장 증거 JSON을 읽어 집계·생성할 뿐이다.

```
cd docs_practice/chapter17
python3 -m venv venv && source venv/bin/activate
pip install -r code/requirements.txt # 외부 의존성 없음
python run_chapter17.py
```

스크립트는 다음 순서로 진행하며, 각 단계가 절 하나에 대응한다.

단계	내용	대응 절
1	앞 장 증거 로드 → 사건 8건 구조화(4분류)	17.1·17.2
2	error budget 계산(4 SLO, 실측 카운터)	17.4
3	성공 요인·실패 패턴 귀납	17.1·17.2
4	조직 운영안 종합(역할·게이트)	17.5
5	포스트모템·재발방지·체크리스트·조직안 마크다운 생성	17.4·17.5

표 17.9 포스트모템 사건 목록(자동 생성 — `ch17_postmortem_report.json`, 8건)

사건 ID	분류	제목	탐지 층	출처 장
INC-08	실제 결함(코드)	materialize 침묵 0건 적재	사용 지점	8장
INC-09	실제 결함(코드)	runs/ 등록 후 로드 실패	로드(읽기)	9장
INC-10	실제 결함(코드)	컨테이너 경로 import 즉시 실패	컨테이너 스모크	10장
INC-12	용량 한계	단일 워커 포화 천장	부하 테스트	12장
INC-14-D1	주입 드릴	오염 메시지 격리	정제 파서	14장
INC-14-D2	주입 드릴	중복 재전송 흡수	먹등 UPSERT	14장
INC-14-D3	주입 드릴	마감 직전 API 중단·복구	부분 실패 격리	14장
INC-15	벤더 장애	OpenAI 생성 전건 429	생성 엔드포인트	15장

여덟 사건이 네 분류로 나뉘는 것에 주목하라 — 같은 “장애”라도 우리 코드의 결함(실제 결함), 계획적으로 주입한 검증(주입 드릴), 자원의 물리적 한계(용량 한계), 외부 제공자의 실패(벤더 장애)는 재발 방지의 방향이 전혀 다르다. 실제 결함은 코드·게이트로, 용량 한계는 확장·용량 계획으로, 드릴은 격리·복구 설계로, 벤더 장애는 폴백으로 대응한다. 분류를 통합하면 재발 방지가 잘못된 층을 대상으로 삼는다.

핵심 코드

error budget 계산은 SLO 목표를 설계값으로 두고, 허용 실패치와 실측 실패를 대조한다.

```
slo_avail = 0.999 # 설계 목표(측정값 아님)
allowed_fail = math.floor(total_req * (1 - slo_avail)) # 허용 예산
budget = {"allowed_failures": allowed_fail, "budget_consumed": total_fail,
          "verdict": "예산 내" if total_fail <= allowed_fail else "예산 초과"}
```

전체 코드는 [docs_practice/chapter17/code/17-1-postmortem.py](#) 참고

결정성 설계

이 장은 새 실행을 하지 않고 앞 장 증거를 집계하므로 `now()` 를 사용하지 않는다 — 타임라인은 사건의 단계 순번으로만 서술한다. 증거 JSON은 `sort_keys=True` 로 재실행해도 바이트 동일하며, 두 번 실행해 `diff` 가 비어 있음을 확인했다. 입력 증거는 앞 장 output에서 `data/input/` 으로 복사한 스냅샷이고 원본을 수정하지 않는다.

실행 중 확인 체크리스트

- ☐ 포스트모템 사건이 8건이고 분류가 실제 결함 3·용량 한계 1·주입 드릴 3·벤더 장애 1인가
- ☐ error budget 요청 가용성이 172633 요청 중 실패 0(허용 172 중 0 소진)인가
- ☐ 예측 성공 예산이 원 3건 소진 후 복구로 잔여 0인가
- ☐ 15장 벤더 장애에서 OpenAI 생성 0·CLOVA 8로 서비스가 지속되는가
- ☐ 산출물 4종(포스트모템·재발방지·체크리스트·조직안)이 생성되고 마지막 줄이 CH17_RUN_PASS 인가
- ☐ 증거 JSON을 재실행해도 바이트 동일한가

산출물 안내

산출물	내용	용도
ch17_postmortem_report.json	사건·성공요인·실패패턴·error budget·조직안 통합 증거	이 장 인용 수치의 증거 파일
ch17_error_budget.json	4 SLO 예산 소진 계산	17.4 표 17.4의 원본
17-1-postmortem.md	포스트모템 보고서(빈 양식을 실측으로 채움)	실습 정보 산출물
ch17_recurrence_prevention_plan.md	재발 방지 계획(사건별 게이트)	지속 개선 자산
ch17_ops_improvement_checklist.md	운영 개선 체크리스트(자동 생성)	운영 점검 자산
ch17_org_operating_plan.md	AI 평가·검증 조직 운영안	17.5 Lab 운영안
code/17-1-postmortem-template.md	빈 포스트모템 양식(정본)	서식 템플릿
표 17.8 사건 원인 가설표(직접 작성)	증상별 근본 원인 가설·탐지 총과 대조 결과	판단 근거 — 3단계 대조 후 확정

실행 결과(실제)

포스트모템: 사건 8건 — 실제 결함(코드) 3, 용량 한계 1, 주입 드릴 3, 벤더 장애 1 — 표 17.2·17.4.
error budget: 요청 가용성 172633 요청·실패 0(허용 172 중 소진 0, 예산 내), 예측 성공 원 3건→복구 후 0, 품질 게이트 채점 2일 중 1일 초과(전체 MAE 1.0 통과), LLM 멀티벤더 CLOVA 8·OpenAI 0.
12장 회수: baseline 2498.3·spike 2788.5 RPS, CPU peak 106.3%, 워커 1→2 1.26배, shadow 0.68배 — 표 17.3.
14장 회수: D2 물리 28→고유 17→중복 11, D3 마감 보류 3→복구 0.
조직 운영안: 역할 4·승인 게이트 3, CH17_RUN_PASS . 증거 JSON 재실행 바이트 동일.

3단계 대조 — 가설과 증거의 대조

표 17.8의 넷째 열을 `ch17_postmortem_report.json`의 사건별 `root_cause · detection_layer`로 채운다. 대조에서 확인할 것 세 가지다.

원인을 코드 밖에서 찾았는가. 네 사건의 근본 원인은 모두 “쓰기·응답은 성공했는데 그 성공이 사용 가능성을 뜻하지 않았다”는 한 형태다(표 17.2). 가설을 “코드 버그”로 적었다면 층을 한 단계 잘못 식별한 것이다 — 각 사건에서 실제로 잘못된 것은 코드가 아니라 **성공의 정의**였다.

탐지 층이 실제 층과 맞았는가. 표 17.9의 “탐지 층” 열이 정답이다. 여기서 자주 어긋나는 것이 9장과 10장이다. 9장은 등록(쓰기)이 아니라 로드(읽기)에서, 10장은 pytest나 호스트 실행이 아니라 컨테이너 스모크에서만 드러났다. 층을 잘못 식별하면 게이트를 잘못된 위치에 설치하게 되고, 그 게이트는 통과하는데 사고는 그대로 발생한다.

분류를 섞지 않았는가. 여덟 사건은 실제 결함 3건·용량 한계 1건·주입 드릴 3건·벤더 장애 1건으로 분류된다. 15장 사건을 “우리 코드 결함”으로 적었다면 재발 방지가 코드 수정으로 향했을 것이나, 실제 대응은 멀티벤더 폴백이다.

설계 선택 두 가지도 실측과 대조한다. 요청 가용성은 172633 요청·실패 0으로 99.9% 목표의 허용 172건 중 한 건도 소진하지 않았다 — 목표를 99.99%로 적었다면 허용은 약 17건이 되지만, 실패가 0이므로 판정은 이 실측에서는 바뀌지 않는다. 즉 이 데이터만으로는 두 목표를 구분할 수 없고, 목표를 결정하는 것은 실측이 아니라 서비스 성격이라는 것이 확인된다. 포스트모템 기준은 자기가 적은 트리거를 8건에 적용해 몇 건이 남는지 계수한다. 8건 모두가 남으면 기준이 느슨한 것이고, 한 건도 남지 않으면 침묵 실패 세 건을 놓치는 기준이다.

표 17.8의 최종본은 이 대조를 반영한 것이며, 과제로 제출하는 것도 이 최종본이다.

관찰

사건 8건이 모두 앞 장 실측에서 유도된다 — 이 실습은 새 장애를 만들지 않고, 이미 발생·기록된 사건을 포스트모템 서식으로 옮겼을 뿐이다.

네 분류(실제 결함·용량 한계·주입 드릴·벤더 장애)를 통합하지 않은 것이 요점이다 — 계획된 드릴을 “실제 사고”로 과장하거나 벤더 장애를 우리 코드 결함으로 오귀속하면 재발 방지가 잘못된 대상을 향한다.

error budget이 순간 소진과 복구 후 잔여를 구분한다(예측 성공 3건→0) — “얼마나 실패했나”와 “얼마나 남았나”는 다른 질문이며, 복구 설계가 후자를 지킨다.

전체 MAE 통과(1.0)가 일자별 초과(2일 중 1일)를 가린 것이 감시 입도의 실측 교훈이다 — 집계 지표만 보면 국소 열화를 놓친다.

운영 개선 체크리스트와 조직 운영안을 수작업으로 쓰지 않고 사건·게이트에서 생성한 것이, 13장 감사표·14장 인수 검수표·16장 모델카드의 “증거에서 자동 생성” 철학의 종합판이다.

실습과 운영의 간극(축소 지점 목록)

사건 수(8건 ↔ 실운영의 수백 건), 자동화(수동 실행 ↔ 사고 발생 시 자동 포스트모템 초안), error budget 정책(계산만 ↔ 소진 시 배포 동결 집행), 조직(운영안 문서 ↔ 실제 인력·책임 배정). 축소된 것은 규모와 집행이지 서식·구조가 아니다 — 포스트모템 서식, 3-why, error budget, 게이트 계보는 어느 규모에서든 같은 형태로 적용된다.

4단계 위험 식별 — 설계와 증거를 검수 언어로 옮기기

앞 세 단계에서 가설(1단계)과 실행 증거(2·3단계)를 모두 확보했다. 검수·감사가 요구하는 형태는 그 둘을 이어 붙인 것이다 — “무엇이 잘못될 수 있는가, 그 판단의 근거가 어느 산출물의 어느 값인가, 무엇으로 막는가”.

위험	실습에서의 근거	대응 방향
사고 은폐(비난 문화)	각 장 “실제로 발생한 오류”를 본문에 수록	비난 없는 포스트모템 서식
사건 오귀속(드릴↔실제↔벤더)	4분류로 명확히 구분	사건 분류를 재발 방지 전에 고정
집계 지표가 국소 열화 은폐	전체 MAE 통과·일자별 초과	감시 입도를 SLO에 명시
단일 벤더 종속	OpenAI 429 중 CLOVA 완주	제공자 독립 코어 + 멀티벤더 폴백
가설 고정으로 인한 오귀속	1단계 가설과 실측 root_cause 의 불일치	원인 확정 전에 탐지 층을 증거로 확인

셋째 열이 “모니터링한다”로 채워지면 그 행은 검수 문서에서 유용하지 않다. 대응 방향에는 **바꿀 설정값이나 바꿀 설계 결정**이 들어가야 한다.

제출물 3종: ① 표 17.8 가설표(3단계 대조를 반영한 최종본), ② `ch17_postmortem_report.json`, ③ 위 위험 식별 표. ①이 판단, ②가 증거, ③이 둘을 이은 검수 자산이다.

보충(전문가 트랙 포인터)

FinOps 기반 비용 모니터링: 구체 단가는 사업자·리전·계약에 따라 다르므로 이 교재는 단가를 고정하지 않는다. 실무에서는 12장식 부하 실측으로 얻은 “인스턴스당 처리량”에 사업자 단가를 곱해 “처리 건당 비용”을 산출하고, FinOps의 Inform→Optimize→Operate 단계로 관리한다. 관찰(shadow)·재시도·멀티벤더 폴백은 모두 비용 항목이므로 예산에 명시적으로 포함한다.

SRE error budget policy: 예산이 소진되면 위험을 늘리는 신규 기능 배포를 동결하고 안정화에 집중하는 것이 error budget policy의 핵심 집행 장치다(예시 정책은 긴급·보안 수정 예외와 에스컬레이션 경로를 함께 규정한다). 이 실습은 소진을 계산까지만 하고 동결을

집행하지 않는다 — 실무에서는 CI/CD에 소진율 임계를 연결해 배포 게이트로 만든다.

장기 운영 조직의 역할 분담과 책임 경계: 연구 조직(평가 기준·벤치마크 방법론 설계)과 운영 조직(게이트 집행·감사·재발 방지)을 분리하되, 포스트모템·error budget 리뷰는 두 조직이 함께 본다. LLM-as-a-judge는 보조이며 채점 주체는 사람·독립 검증이다(15·16장).

핵심 용어

포스트모템: 사고의 기록·영향·조치·근본 원인·재발 방지를 담은 문서 — 비난 없이(blameless)

3-why: 왜를 세 번 물어 증상 아래의 구조적 근본 원인에 이르는 분석

error budget: SLO 목표와 실측의 차이 — 허용된 불완전의 예산(SRE)

SLO(Service Level Objective): 조직이 정하는 신뢰성 목표(설계값, 측정값 아님)

소진율(burn rate): 예산을 얼마나 빨리 소진하는가 — 실측 카운터에서 계산

FinOps: 엔지니어링·재무·사업이 함께 비용을 데이터로 판단하는 실천

용량 계획: 필요 처리량을 실측 천장(인스턴스당 RPS)으로 나눠 자원 규모를 정함

침묵 실패: 비어 있거나 파손된 결과에 실패를 알리는 표시가 남지 않는 실패 패턴

주입 드릴(fault injection drill): 계획적으로 장애를 주입해 격리·복구를 검증

벤더 장애: 외부 제공자 측 실패(예: 결제 비활성 429) — 멀티벤더 폴백으로 대응

Policy Methodology Lab: 평가 기준·벤치마크·승인 게이트를 관리하는 상설 조직

책임 경계: 연구 조직(기준 설계) ↔ 운영 조직(게이트 집행)의 분리

검수 질문 한 줄 요약

발주·검수 문서에서 이 장의 내용은 다섯 질문으로 압축된다: ① 안정성의 근거가 사람이 아니라 구조(증화·먹등·결정성·격리)인가, ② 게이트의 기준이 “실행 성공”이 아니라 “사용 가능”인가, ③ 용량 계획이 부하 실측 천장에 근거하는가, ④ 장애가 비난 없이 기록되고 error budget으로 관리되는가, ⑤ 평가 기준·벤치마크·승인 게이트가 상설 조직의 관리 대상인가. 다섯 모두 답이 없으면 “잘 운영한다”는 문장은 구호일 뿐이다.

전권 마무리

1~2장에서는 데이터의 위험을 다루었다. 3~7장에서는 데이터 처리 파이프라인을 구현했고, 8~11장에서는 피처와 모델의 등록·배포·감시 절차를 다루었다. 12~14장에서는 부하 시험·거버넌스·통합 운영을 검증했고, 15~16장에서는 LLM과 윤리·규제 요구사항을 추가했다. 17장에서는 이 결과를 바탕으로 성공 요인과 실패 패턴을 분석하고, 비용·사고·조직 관리 절차를 정리했다. 이 교재가 실제 오류를 본문에 포함한 목적은 실패 원인을 기록하고 검증 게이트와 재발 방지 절차로 반영하는 데 있다.

참고문헌

- Lunney, J., & Lueder, S. (2016). *Postmortem Culture: Learning from Failure*. In *Site Reliability Engineering* (Google SRE Book). <https://sre.google/sre-book/postmortem-culture/>
- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Embracing Risk* (Error Budgets). In *Site Reliability Engineering* (Google SRE Book). <https://sre.google/sre-book/embracing-risk/>
- FinOps Foundation. (n.d.). *FinOps Framework — Overview, Principles & Phases*. <https://www.finops.org/framework/> (principles: <https://www.finops.org/framework/principles/> · phases: <https://www.finops.org/framework/phases/>)
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., et al. (2015). *Hidden Technical Debt in Machine Learning Systems*. NeurIPS. <https://proceedings.neurips.cc/paper/2015/hash/86df7dcfd896fcdf2674f757a2463eba-Abstract.html>
- Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2017). *The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction*. IEEE Big Data. <https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production-readiness-and-technical-debt-reduction/>
- Little, J. D. C. (1961). *A Proof for the Queuing Formula: $L = \lambda W$* . Operations Research, 9(3). <https://doi.org/10.1287/opre.9.3.383>
- 법제처. *개인정보 보호법* (제18조 목적 외 이용·제공 제한). <https://www.law.go.kr/법령/개인정보보호법>
- Google. *Example Error Budget Policy*. In *The Site Reliability Workbook* (Appendix B). <https://sre.google/workbook/error-budget-policy/>
- PagerDuty. *Postmortem Process* (Blameless Postmortems). PagerDuty Incident Response Documentation. https://response.pagerduty.com/after/post_mortem_process/
- NIST. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0) — Govern* function. <https://www.nist.gov/itl/ai-risk-management-framework>
- DORA (DevOps Research and Assessment). *Research & Capabilities*. <https://dora.dev/research/>